

URL Kısaltıcı Servisi

Backend Staj Projesi — Proje Brifi

Bu projede, uzun bir URL'yi kısa bir kodla eşleştiren, kısa kod ziyaret edildiğinde orijinal adrese yönlendiren ve her linkin tıklanma istatistiğini tutan bir REST API geliştireceksin. Kayıtlı kullanıcılar kendi linklerini oluşturup yönetebilecek. Amaç; REST API tasarımı, kimlik doğrulama ve veri modelleme konularında uçtan uca bir uygulama ortaya koymak.

Teknik Serbestlik

Dil ve framework tamamen sana kalmış (Node.js/Express, Python/FastAPI, Java/Spring, Go — hangisiyle rahatsan). Veritabanı olarak PostgreSQL, MySQL veya SQLite kullanabilirsin. Seçimlerinin gerekçesini README'de kısaca açıklaman yeterli.

Fonksiyonel Gereksinimler

Kimlik Doğrulama

- Kullanıcı kaydı: e-posta ve parola ile.
- Giriş yapıldığında JWT token dönmeli.
- Parolalar asla düz metin saklanmamalı; bcrypt veya argon2 ile hash'lenmeli.
- Korumalı uç noktalar `Authorization: Bearer <token>` başlığı beklemeli.

Link Yönetimi (giriş gerektirir)

- Yeni kısa link oluşturma: uzun URL verilir, kısa kod döner.
- Kullanıcının kendi linklerini listeleyebilmesi.
- Link silme — yalnızca linkin sahibi silebilmeli.
- İsteğe bağlı: kullanıcının kendi belirlediği özel alias (çakışma kontrolü ile).

Yönlendirme (herkese açık)

- `GET /{kod}` orijinal URL'ye yönlendirmeli (301/302).
- Kod bulunamazsa 404 dönmeli.

İstatistik (giriş gerektirir)

- Her linkin toplam tıklanma sayısı.
- Bonus: son 7 günün günlük dağılımı ya da referrer/user-agent kırılımı.

Önerilen Uç Noktalar

Method	Endpoint	Auth	Açıklama
POST	/auth/register	Hayır	Kayıt
POST	/auth/login	Hayır	Giriş, JWT döner
POST	/links	Evet	Kısa link oluşturun
GET	/links	Evet	Kendi linklerinizi listele
GET	/links/{id}/stats	Evet	Link istatistiği
DELETE	/links/{id}	Evet	Link sil
GET	/{kod}	Hayır	Yönlendirme + tıklama kaydı

Önerilen Veri Modeli

- users:** id, email, password_hash, created_at
- links:** id, user_id (FK), short_code, original_url, created_at
- clicks:** id, link_id (FK), clicked_at, referrer, user_agent

Teknik Beklentiler

- Kısa kod üretimi çakışmasız olmalı (base62, nanoid veya benzeri bir yaklaşım).
- Girdi doğrulaması yapılmalı; geçersiz URL reddedilmeli.
- Anlamlı HTTP durum kodları kullanılmalı (200 / 201 / 400 / 401 / 404).
- Hatalar tutarlı bir JSON formatında dönmeli, örneğin `{ "error": "..."}` .
- Yapılandırma `.env` üzerinden okunmalı; JWT secret ve DB bilgileri kodun içine gömülmemeli.

- README kurulum adımlarını ve örnek istekleri (curl veya Postman) içermeli.

Kabul Kriterleri

- ☐ Kayıt ve giriş çalışıyor, token dönüyor.
- ☐ Parolalar hash'li saklanıyor.
- ☐ Auth gerektiren uç noktalar token olmadan 401 dönüyor.
- ☐ Link oluşturuluyor ve yönlendirme doğru çalışıyor.
- ☐ Başka bir kullanıcının linki görüntülenemiyor/silinemiyor.
- ☐ Tıklama sayısı doğru artıyor.
- ☐ Geçersiz URL veya bozuk istek düzgün hata dönüyor.
- ☐ README ile proje sıfırdan ayağa kaldırılabiliyor.

Bonus (zaman kalırsa)

- Rate limiting: aynı IP'den belirli süre içinde sınırlı istek.
- Kısa kod için QR kod döndüren bir uç nokta.
- Auth ve kod üretimi için birkaç birim test.
- Linklere son kullanma tarihi (expiry) eklenmesi.

Sorularını çekinmeden ilet. Değerlendirme; kodun çalışırılığı, temiz yapı, güvenlik pratikleri ve dokümantasyon üzerinden yapılacak.